

EX. NO: 8**AVL TREES****DATE:****QUESTION :**

Implement creation of an AVL Tree with insertion and deletion operations and display both the an AVL Tree with insertion and deletion operations.

AIM:

To implement creation of an AVL Tree with insertion and deletion operations and display both the an AVL Tree with insertion and deletion operations.

ALGORITHM:

1. Start.
2. Initialize the AVL tree with root = NULL.
3. Read the number of elements and insert each element into the tree using BST insertion.
4. For every insertion, create a new node if the tree is empty or recursively insert the value into the left or right subtree.
5. Update the height of each node and calculate its Balance Factor using:
Balance Factor = Height of Left Subtree – Height of Right Subtree
6. Check whether the node is unbalanced and perform the appropriate rotation:
 - o LL Case: Perform Right Rotation.
 - o RR Case: Perform Left Rotation.
 - o LR Case: Perform Left Rotation and then Right Rotation.
 - o RL Case: Perform Right Rotation and then Left Rotation.
7. For Search, compare the given value with the current node and recursively search the left or right subtree.
8. Perform the required tree traversal:
 - o Inorder: Left → Root → Right
 - o Preorder: Root → Left → Right
 - o Postorder: Left → Right → Root
9. Display the selected traversal or all traversals of the AVL tree.
10. Repeat the operations using the menu until the user selects Exit.
11. Before exiting, free the memory allocated for all nodes.
12. Stop.

PROGRAM:

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    int height;
    struct Node *left;
    struct Node *right;
};

/* Get height of a node */
int height(struct Node *root) {
    if (root == NULL)
        return 0;

    return root->height;
}

/* Find maximum of two numbers */
int max(int a, int b) {
    return (a > b) ? a : b;
}

/* Create a new node */
struct Node* createNode(int value) {
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->height = 1;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

/* Right Rotation */
struct Node* rightRotate(struct Node *y) {
    struct Node *x;
    struct Node *t;

    x = y->left;
    t = x->right;

    x->right = y

```

```

y->left = t;

y->height = 1 + max(height(y->left), height(y->right));
x->height = 1 + max(height(x->left), height(x->right));

return x;
}

/* Left Rotation */
struct Node* leftRotate(struct Node *x) {
    struct Node *y;
    struct Node *t;

    y = x->right;
    t = y->left;

    y->left = x;
    x->right = t;

    x->height = 1 + max(height(x->left), height(x->right));
    y->height = 1 + max(height(y->left), height(y->right));

    return y;
}

/* Get Balance Factor */
int getBalance(struct Node *root) {
    if (root == NULL)
        return 0;

    return height(root->left) - height(root->right);
}

/* Insert a node into AVL Tree */
struct Node* insert(struct Node *root, int value) {

    /* Normal BST insertion */
    if (root == NULL)
        return createNode(value);

    if (value < root->data) {
        root->left = insert(root->left, value);
    }
    else if (value > root->data) {
        root->right = insert(root->right, value);
    }
    else {
        /* Duplicate values are not allowed */
        return root;
    }

    /* Update height */
    root->height = 1 + max(height(root->left),

```

```

        height(root->right));

/* Get balance factor */
int balance = getBalance(root);

/* LL Case */
if (balance > 1 && value < root->left->data) {
    return rightRotate(root);
}

/* RR Case */
if (balance < -1 && value > root->right->data) {
    return leftRotate(root);
}

/* LR Case */
if (balance > 1 && value > root->left->data) {
    root->left = leftRotate(root->left);
    return rightRotate(root);
}

/* RL Case */
if (balance < -1 && value < root->right->data) {
    root->right = rightRotate(root->right);
    return leftRotate(root);
}

return root;
}

/* Search an element */
void search(struct Node *root, int value) {

    if (root == NULL) {
        printf("Element not found\n");
        return;
    }

    if (root->data == value) {
        printf("Element found\n");
        return;
    }

    if (value < root->data)
        search(root->left, value);
    else
        search(root->right, value);
}

/* Inorder Traversal */
void inorder(struct Node *root) {

    if (root != NULL) {
        inorder(root->left);

```

```

        printf("%d ", root->data);
        inorder(root->right);
    }
}

/* Preorder Traversal */
void preorder(struct Node *root) {

    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

/* Postorder Traversal */
void postorder(struct Node *root) {

    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

/* Display all traversals */
void display(struct Node *root) {

    if (root == NULL) {
        printf("AVL Tree is empty\n");
        return;
    }

    printf("\nInorder  : ");
    inorder(root);

    printf("\nPreorder : ");
    preorder(root);

    printf("\nPostorder : ");
    postorder(root);

    printf("\n");
}

/* Free all nodes */
void freeTree(struct Node *root) {

    if (root != NULL) {
        freeTree(root->left);
        freeTree(root->right);
        free(root);
    }
}

```

```

/* Main Function */
int main() {

    struct Node *root = NULL;

    int choice;
    int value;
    int n;
    int i;

    while (1) {

        printf("\n===== \n");
        printf("    AVL TREE MENU\n");
        printf("===== \n");
        printf("1. Create AVL Tree\n");
        printf("2. Insert\n");
        printf("3. Search\n");
        printf("4. Inorder Traversal\n");
        printf("5. Preorder Traversal\n");
        printf("6. Postorder Traversal\n");
        printf("7. Display\n");
        printf("8. Exit\n");
        printf("===== \n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {

            /* Create AVL Tree */
            case 1:

                /* Clear existing tree */
                if (root != NULL) {
                    freeTree(root);
                    root = NULL;
                }

                printf("Enter number of elements: ");
                scanf("%d", &n);

                if (n <= 0) {
                    printf("Invalid number of elements!\n");
                    break;
                }

                printf("Enter %d elements:\n", n);

                for (i = 0; i < n; i++) {
                    scanf("%d", &value);

                    root = insert(root, value);
                }
            }
        }
    }
}

```

```
    }

    printf("AVL Tree created successfully!\n");

    break;

/* Insert */
case 2:

    printf("Enter value to insert: ");
    scanf("%d", &value);

    root = insert(root, value);

    printf("Element inserted successfully!\n");

    break;

/* Search */
case 3:

    if (root == NULL) {
        printf("AVL Tree is empty\n");
    }
    else {
        printf("Enter value to search: ");
        scanf("%d", &value);

        search(root, value);
    }

    break;

/* Inorder */
case 4:

    if (root == NULL) {
        printf("AVL Tree is empty\n");
    }
    else {
        printf("Inorder Traversal: ");
        inorder(root);
        printf("\n");
    }

    break;

/* Preorder */
case 5:

    if (root == NULL) {
        printf("AVL Tree is empty\n");
    }
    else {
```

```
        printf("Preorder Traversal: ");
        preorder(root);
        printf("\n");
    }

    break;

/* Postorder */
case 6:

    if (root == NULL) {
        printf("AVL Tree is empty\n");
    }
    else {
        printf("Postorder Traversal: ");
        postorder(root);
        printf("\n");
    }

    break;

/* Display */
case 7:

    display(root);

    break;

/* Exit */
case 8:

    freeTree(root);

    printf("Program exited successfully.\n");

    return 0;

/* Invalid choice */
default:

    printf("Invalid choice! Please try again.\n");
}
}

return 0;
}
```


INPUT AND OUTPUT:

```
[ita2536@mritlin harshan]$ vi ex8.c
[ita2536@mritlin harshan]$ cc ex8.c
[ita2536@mritlin harshan]$ ./a.out
```

```
=====
                        AVL TREE MENU
=====
```

1. Create AVL Tree
2. Insert
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit

```
=====
Enter your choice: 1
Enter number of elements: 7
Enter 7 elements:
```

```
1
2
3
4
5
6
7
```

```
AVL Tree created successfully!
```

```
=====
                        AVL TREE MENU
=====
```

1. Create AVL Tree
2. Insert
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit

```
=====
Enter your choice: 3
Enter value to search: 5
Element found
```

```
=====
              AVL TREE MENU
=====
1. Create AVL Tree
2. Insert
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
=====
Enter your choice: 4
Inorder Traversal: 1 2 3 4 5 6 7

=====
              AVL TREE MENU
=====
1. Create AVL Tree
2. Insert
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
=====
Enter your choice: 5
Preorder Traversal: 4 2 1 3 6 5 7

=====
              AVL TREE MENU
=====
1. Create AVL Tree
2. Insert
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
=====
Enter your choice: 6
Postorder Traversal: 1 3 2 5 7 6 4
```

```
=====
                        AVL TREE MENU
=====
1. Create AVL Tree
2. Insert
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
=====
Enter your choice: 7

Inorder   : 1 2 3 4 5 6 7
Preorder  : 4 2 1 3 6 5 7
Postorder : 1 3 2 5 7 6 4

=====
                        AVL TREE MENU
=====
1. Create AVL Tree
2. Insert
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
=====
Enter your choice: 8
Program exited successfully.
[ita2536@mritlin harshan]$
```

RESULT:

Thus the implementation to create an AVL Tree with insertion and deletion operations and display both the an AVL Tree with insertion and deletion operations was executed successfully.